Ludovic BERGERON

Fine-Tuning QLoRA d'un LLM

Application au coach IA de Loodo

Paramètre	Valeur
Modèle de base	Mistral 7B Instruct v0.3 (QLoRA 4-bit)
Méthode	QLoRA + LoRA (rank=16, alpha=32)
Framework	Unsloth 2026.3.5 + TRL + HuggingFace Transformers
GPU utilisé	Tesla T4 — 15.6 Go VRAM (Google Colab)
Application cible	Loodo — coach motivation/organisation personnel
Modèle publié	HuggingFace Hub — loodo-mistral-7b-coach-fr

1. Contexte et objectif

Loodo est une application mobile de productivité et de motivation personnelle, développée en Flutter/Supabase. L'application intègre un assistant IA de coaching dont l'objectif est d'aider les utilisateurs à surmonter la procrastination, gérer leur temps et maintenir leur motivation.

L'objectif de ce projet Fine-Tuning est de spécialiser un Large Language Model (LLM) open-source, Mistral 7B, pour qu'il adopte la personnalité et le style de réponse de Loodo : direct, bienveillant, actionnable, toujours orienté vers une micro-action concrète immédiate.

Ce projet montre la pipeline complète de fine-tuning : préparation du dataset, configuration LoRA, entraînement, évaluation qualitative et quantitative, et publication du modèle.

2. Concepts clés

2.1 Fine-tuning

Un LLM pré-entraîné comme Mistral 7B a été exposé à des milliards de tokens lors de son entraînement initial. Le fine-tuning consiste à reprendre ce modèle et à continuer l'entraînement sur un dataset spécialisé et beaucoup plus petit, pour que le modèle adopte un comportement précis. Plutôt que de partir de zéro, on ajuste les poids existants.

2.2 LoRA — Low-Rank Adaptation

Le problème du fine-tuning classique est la mémoire : Mistral 7B possède 7 milliards de paramètres. Les modifier tous demande 60+ Go de VRAM, impossible sur un GPU grand public.

LoRA résout ce problème élégamment : au lieu de modifier les poids originaux W, on les gèle et on ajoute à côté deux petites matrices entraînables A et B telles que :

```
ΔW = A × B    (où rank r << d)
```

Avec r=16, on entraîne ~42 millions de paramètres au lieu de 7 milliards, soit 1.10% du modèle. Le fichier adapter résultant fait ~100 Mo au lieu de 14 Go.

2.3 QLoRA — Quantized LoRA

QLoRA ajoute une dimension supplémentaire : avant d'appliquer LoRA, le modèle de base est chargé en 4-bit (format NF4) au lieu de 16-bit. Mistral 7B passe ainsi de ~14 Go à ~4 Go en VRAM.

Approche	VRAM nécessaire	Faisable sur T4
Fine-tuning complet 16-bit	~60 Go	Non
LoRA 16-bit	~20 Go	Non
QLoRA 4-bit (ce projet)	~8 Go	Oui

3. Préparation du dataset

3.1 Choix et justification

Aucun dataset public de coaching motivation/organisation en français n'existait. La solution retenue est la génération de données synthétiques par LLM, méthode utilisée en industrie (cf. Stanford Alpaca, Self-Instruct).

3.2 Format

Chaque exemple suit le format ChatML natif de Mistral, composé de trois messages :

- system : définit la personnalité de Loodo (coach direct, bienveillant, actionnable)
- user : la situation de l'utilisateur (procrastination, surcharge, découragement...)
- assistant : la réponse coaching attendue (micro-action, reformulation, plan)

3.3 Évolution du dataset

Version	Exemples	Scénarios couverts	Méthode
V1	51	Procrastination, surcharge, concentration, habitudes, stress	Généré par Claude Sonnet 4.6
V2	150	V1 + perfectionnisme, comparaison sociale, fatigue, blocage créatif, objectifs flous	Généré par Chatgpt-5.4

Note : en production, 500–1000+ exemples seraient nécessaires pour un modèle robuste.

4. Pipeline d’entraînement

4.1 Stack technique

- Unsloth 2026.3.5 — kernels CUDA optimisés, 2x plus rapide que l’approche standard
- HuggingFace Transformers 5.3.0 — chargement et tokenisation du modèle
- PEFT — implémentation de LoRA
- TRL SFTTrainer — boucle d’entraînement supervisé (Supervised Fine-Tuning)
- bitsandbytes — quantification 4-bit NF4
- Google Colab — GPU Tesla T4 (15.6 Go VRAM)

4.2 Configuration LoRA

Paramètre	Valeur	Justification
rank (r)	16	Bon compromis apprentissage/mémoire pour 150 exemples
lora_alpha	32	Convention : alpha = 2 × r pour scaling optimal
target_modules	q/k/v/o_proj + gate/up/down_proj	Couches attention + feed-forward — couvre toute la mécanique
lora_dropout	0	Dataset petit, pas besoin de régularisation supplémentaire
use_gradient_checkpointing	unsloth	Économise ~30% VRAM en recalculant les activations

4.3 Hyperparams d’entraînement

Paramètre	V1	V2	Justification
learning_rate	2e-4	1e-4	V2 part de poids déjà ajustés, lr plus faible évite la déstabilisation
batch_size effectif	8 (2×4)	8 (2×4)	gradient_accumulation simule un batch plus grand sans surcharge VRAM
epochs	3	3	Suffisant pour 51/150 exemples sans overfitting
warmup_steps	5	10	Dataset plus grand, on peut se permettre un warmup plus long
optimizer	adamw_8bit	adamw_8bit	Version 8-bit d’AdamW, réduit la VRAM des moments optimizer
lr_scheduler	linear	linear	Learning rate décroit linéairement après le warmup

5. Résultats et évaluation

5.1 Métriques quantitatives

Métrique	Baseline	V1 (51 ex.)	V2 (150 ex.)
Loss finale	—	0.9451	0.4872
Amélioration loss	—	référence	-48% vs V1
Temps entraînement	—	1m54s	4m57s
Steps	—	21	57
VRAM peak	—	Non mesurée	7.75 Go (49.6%)
Vitesse	—	Non mesurée	1.48 ex./sec
Paramètres entraînés	0	41.9M (1.10%)	41.9M (1.10%)

5.2 Évaluation qualitative

Question test : « J’arrive pas à me concentrer depuis 2 heures, je procrastine sur tout. »

Baseline (Mistral 7B sans fine-tuning)

> « Cela peut être dû à plusieurs facteurs, mais l'une des causes les plus courantes est l'anxiété chronique. Vous pouvez essayer de vous poser des questions telles que "Qu'est-ce qui m'empêche de me concentrer ?", "Quelle est la pire chose qui peut arriver si je ne suis pas distrait maintenant ?" et "Quand je suis concentré, je peux réussir à accomplir ce qui est important pour moi. Quand je suis distrait, je me sens perdu et je n'ai pas de réussite". En répondant à ces questions, vous pouvez comprendre les causes de votre distraction. »
> Observation : réponse générique, clinique, théorique. Pas de micro-action. Ton formel.

Après fine-tuning V1 (51 exemples)

> «Procrastination c'est souvent le signe que le cerveau a besoin de direction. On va créer un plan d'action de 2 minutes, pas un plan de 2 heures — le cerveau ne s'engage pas avec ça. Dis-moi ce que tu dois faire aujourd'hui en 3 mots maximum. Ensuite, on découpe ce incompletable en 3 à 5 actions concrètes de 5 minutes chacune. On part de l'imprévisible au prédicable. Par exemple : 'Rédiger le mémo pour la réunion demain' devient 'Ouvre Google Docs, écrit le titre, développe le point 1 en 2 lignes, enregistre et ferme — 5 minutes.' Le cerveau se dit 'je peux ça'. »
> Observation : ton Loodo reconnaissable, micro-action proposée, structure claire. Amélioration nette.

Après fine-tuning V2 (150 exemples)

> 2 heures de perte de concentration, c'est souvent le signal que ton cerveau a besoin d'une pause. Pas d'accuse, pas de culpabilité — tu as travaillé avant. Fais une micro-pause de 5 minutes : marche, station debout, stretching, respiration. Puis relance sur une micro-tâche précise de moins de 5 minutes. Travaller fatigué te coûte plus de temps au final. »
> Observation : plus concis, plus fluide, plus empathique. Hallucination résiduelle (« tu as travaillé avant » — voir section 5.3).

5.3 Limites et analyse des hallucinations

La réponse V2 contient une hallucination notable : le modèle affirme « tu as travaillé avant » alors que l'utilisateur n'a rien dit de tel.

Explication : le modèle a appris statistiquement que la procrastination survient souvent après une période de travail. Il généralise ce pattern à tous les cas, inventant du contexte absent de la conversation. C'est une limite fondamentale du fine-tuning sur peu de données : avec 150 exemples, la diversité des situations est insuffisante pour que le modèle apprenne à rester strictement dans le contexte fourni.

Solution : augmenter à 500+ exemples avec des cas où l'utilisateur précise explicitement ne pas avoir travaillé, ce qui apprendrait au modèle à conditionner sa réponse au contexte réel.

6. Analyse GPU

6.1 Pourquoi un GPU ?

Le fine-tuning d'un LLM repose massivement sur des opérations matricielles : multiplication de matrices, calcul de gradients, mise à jour de poids. Ces opérations sont intrinsèquement parallélisables, exactement ce pour quoi les GPU ont été conçus.

Un CPU exécute ces calculs de manière séquentielle avec quelques dizaines de cœurs. Le Tesla T4 dispose de 2560 cœurs CUDA capables de traiter des milliers d'opérations en parallèle, réduisant le temps d'entraînement de plusieurs ordres de grandeur.

6.2 Spécifications du GPU utilisé

Spécification	Valeur
Modèle	NVIDIA Tesla T4
Architecture CUDA	Turing (compute 7.5)
VRAM totale	15.6 Go GDDR6
CUDA Cores	2560
CUDA Toolkit	12.8
Précision supportée	FP32, FP16, INT8, INT4
Bfloat16	Non (pas disponible sur Turing — utilise FP16)
xFormers	0.0.35 (attention optimisée)

6.3 Métriques GPU mesurées

Métrique	V1	V2
VRAM peak utilisée	Non mesurée	7.75 Go
Utilisation VRAM	Non mesurée	49.6% (7.75 / 15.6 Go)
Temps d'entraînement	1m54s	4m57s
Vitesse	Non mesurée	1.48 exemples/sec
Steps/sec	Non mesurée	0.19 steps/sec
Nombre de steps	21	57
Paramètres entraînés	41.9M	41.9M
Ratio LoRA	1.10%	1.10%

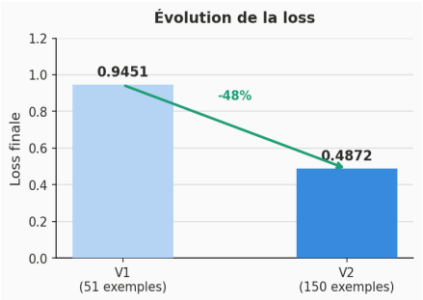

Fig. 1 — Évolution de la loss V1 → V2

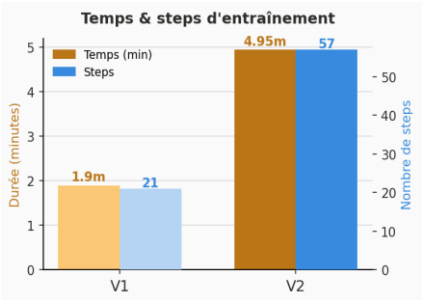

Fig. 2 — Temps d'entraînement et steps

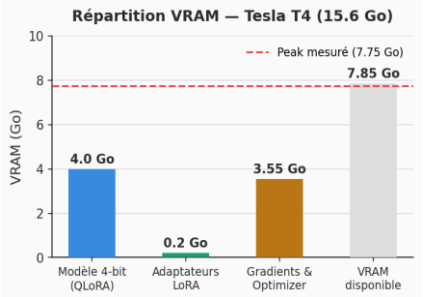

Fig. 3 — Répartition VRAM pendant l'entraînement

6.4 Optimisations mémoire

Plusieurs techniques ont été combinées pour faire tenir l'entraînement de Mistral 7B dans les 15.6 Go du T4 :

Technique	Impact VRAM	Description
Quantification 4-bit (QLoRA)	-65%	Modèle de base : 14 Go → ~4 Go (Fig. 4)
Gradient checkpointing	-30%	Recalcul des activations au lieu de les stocker
AdamW 8-bit	-50% optimizer	Moments optimizer compressés de 32-bit à 8-bit
Gradient accumulation (steps=4)	Neutre	Simule batch=8 sans charger 8 exemples simultanément
LoRA (r=16)	-99% paramètres	1.10% des paramètres entraînés au lieu de 100%

Résultat combiné : VRAM peak de 7.75 Go pour un modèle de 7 milliards de paramètres, soit une réduction de ~95% par rapport au fine-tuning standard 16-bit.

6.5 Analyse : dataset vs GPU

Le passage de V1 à V2 (dataset x3) a multiplié le temps d'entraînement par 2.6 (1m54s → 4m57s), proportionnellement au nombre de steps (21 → 57). La VRAM utilisée reste stable : le modèle et la config LoRA sont identiques, seule la durée augmente.

> *Tripler les données a divisé la loss par 2 (0.94 → 0.48), pour seulement 3 minutes de GPU supplémentaires. En fine-tuning, le dataset est le levier principal — bien avant les hyperparams.*
> Conclusion principale du projet

7. Conclusion

Ce projet démontre l'intégralité du pipeline de fine-tuning d'un LLM en conditions réelles :

- Génération d'un dataset synthétique spécialisé (méthode Alpaca)
- Fine-tuning QLoRA sur GPU avec Unsloth
- Démarche expérimentale V1 → V2 avec métriques
- Évaluation qualitative avec analyse des limites (hallucinations)
- Publication du modèle sur HuggingFace Hub avec model card

La principale leçon technique : avec un dataset x3 plus grand et seulement 3 minutes de GPU supplémentaires, la loss a été divisée par deux. En fine-tuning, la qualité et la quantité des données ont plus d'impact que l'optimisation des hyperparams.

Pistes d'amélioration : augmenter à 500+ exemples, introduire des conversations multi-tours, et expérimenter avec DPO (Direct Preference Optimization) pour affiner le style de réponse au-delà du SFT.

Liens du repo GitHub : Pirodax/Coach-fr-mistral-fine-tuning
Liens du Model HuggingFace : Pirodax/loodo-mistral-7b-coach-fr · Hugging Face